

## **Reflective Learning Journal**

Project: Smart Traffic Management Edge AI

Hasan Zaidi

Prof. Ayyagari

ITAI 3377

April 30, 2026

**Purpose of the Project** The objective of this capstone project was to design, develop, and deploy an AI-driven autonomous agent within an Industrial Internet of Things (IIoT) environment. I chose the practical path, focusing on a Smart Traffic Management system. The goal was to simulate an edge device that collects real-time vehicle density data at an intersection, processes that data locally using a trained AI model, and autonomously adjusts traffic light cycles to optimize traffic flow.

**Concepts Covered** This project tied together several advanced concepts in AI and edge computing:

- **Edge AI Deployment:** Training a machine learning model using TensorFlow and converting it into a lightweight **.tflite** format for local execution.
- **Autonomous Decision Making:** Writing a Python script (`edge_traffic_agent.py`) that uses **ai-edge-litert** to interpret sensor data and output confidence-backed decisions, for example, **GREEN\_NS (Confidence: 0.90)**.
- **IIoT Telemetry & Messaging:** Utilizing the MQTT protocol to transmit secure, real-time JSON payloads from the edge agent to a central system.
- **Data Visualization:** Configuring a Node-RED dashboard to subscribe to the MQTT topic (**smart\_city/traffic/intersection\_1**) and visually parse the AI's autonomous commands.

**Challenges and Errors Encountered** Building this system required significant troubleshooting, which provided valuable hands-on experience with environment management and networking:

1. **Library Compatibility:** I initially attempted to use the standard **tflite-runtime** library, but it failed to install due to conflicts with my newer version of Python on macOS. I

overcame this by researching and migrating my code to use Google's newer **ai-edge-litert-nightly** package.

2. **Execution Environments:** I encountered a **ValueError** stating the script could not open **model.tflite**. This happened because I was trying to run the local edge agent script inside the cloud-based Google Colab environment. I resolved this by properly separating the workflow: training and converting the model in Colab, then downloading the **.tflite** file to my local Mac directory to run alongside the Python agent.
3. **MQTT Broker Connection Refusals:** When running the final agent, I received a **ConnectionRefusedError: [Errno 61]**. This occurred because the local Mosquitto broker was not actively running in the background on my Mac. Additionally, macOS networking quirks caused issues with standard "localhost" routing. I resolved this by manually starting the Mosquitto service via Homebrew and explicitly changing the broker IP in my Python code to **127.0.0.1**.

**Insights and Conclusion** Once the environment and networking issues were resolved, the system worked seamlessly. Watching the terminal successfully log the AI's autonomous decisions (e.g., **Action Executed: GREEN\_EW (Confidence: 0.80)**) and seeing those actions extract instantly into the Node-RED flow was highly rewarding. While this project used simulated sensor data to prove the architecture, the framework is fully functional. A future iteration of this project would involve replacing the Python simulation with live camera feeds or an ESP32 hardware setup to capture real-world traffic data.